

NLP (NATURAL LANGUAGE PROCESSING)

Computer Language

Computer languages are languages used to interact with a computer, such as Python, C++, Java, HTML, etc.

Can computers understand our language?

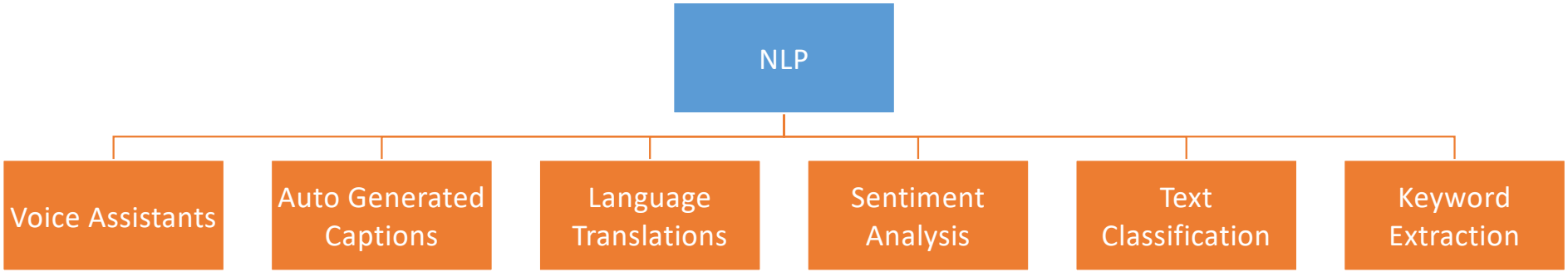
Computers require a specific set of instructions to understand human input called programs. To talk toa computer, we convert natural language into a language that a computer understands. We need Natural Language Processing to help computers understand natural language.



Why NLP is Important?

The whole purpose of NLP is to *make communication between computer systems and humans possible*. This includes creating different tools and techniques that facilitate better communication of intent and context.

Applicatios of NLP



Keyword Extraction



Purpose: To learn how to utilize an API for performing keyword extraction from a website.

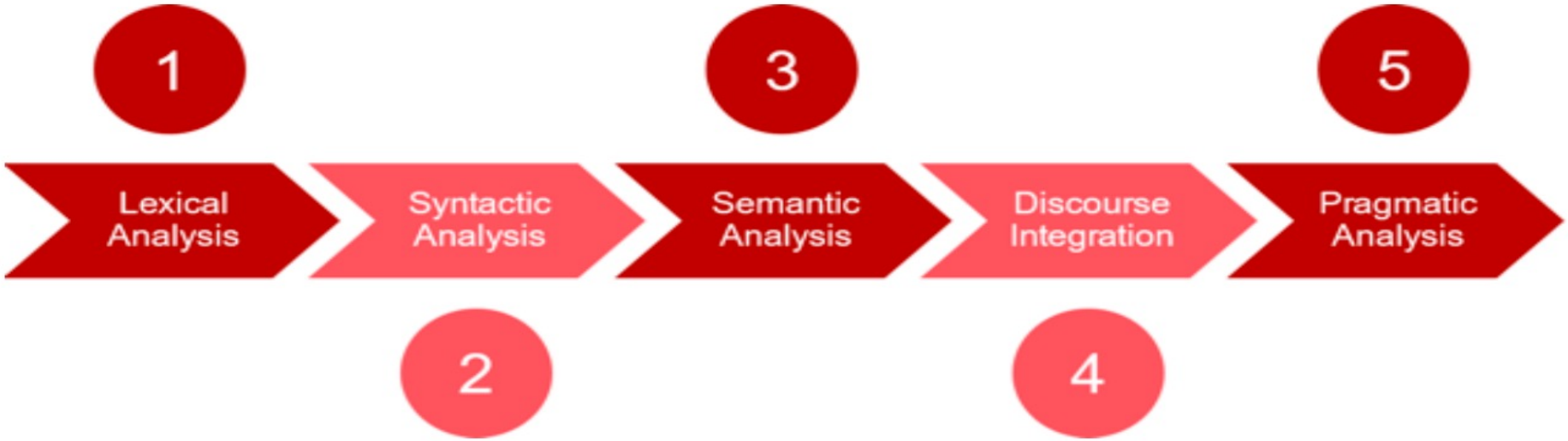
Say: “Keyword extraction in NLP involves automatically identifying and extracting the most important words or phrases from a piece of text. These keywords represent the main topics or themes within the text and are useful for tasks like document summarization, information retrieval, and contentanalysis.”

https://cloud.google.com/natural-language

Stages of NLP

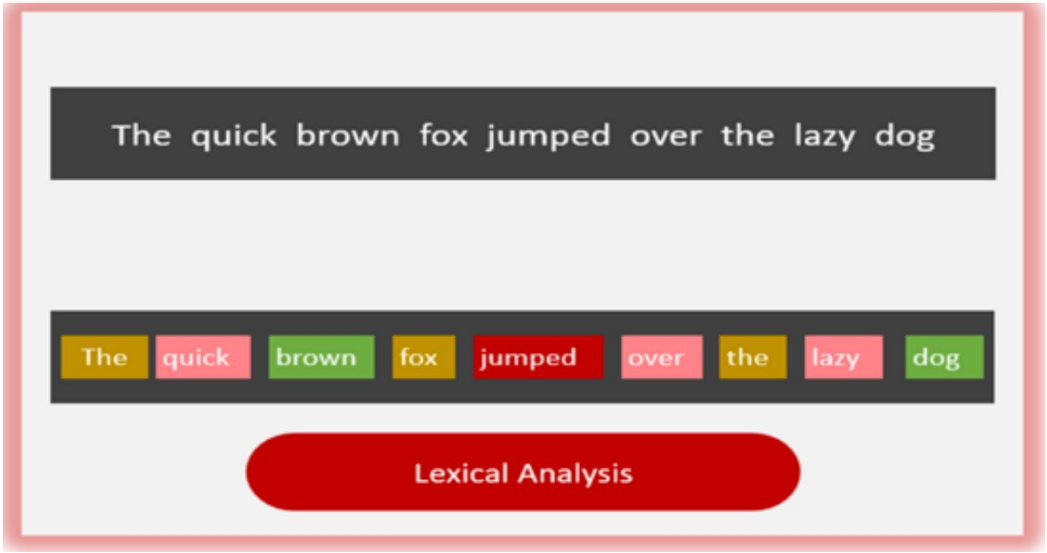


The different stages of Natural Language Processing (NLP)serve various purposes in the overall task of understanding and processing human language. The stages of Natural Language Processing (NLP) typically involve the following:



Lexical Analysis

NLP starts with identifying the structure of input words. It is the process of dividing a large chunk of words into structural paragraphs, sentences, and words. Lexicon stands for a collection of the various words and phrases used in a language.



Lengthy text is broken down into chunks.

Syntactic Analysis / Parsing

It is the process of *checking the grammar of sentences and phrases*. It forms a relationship among words and eliminates logically incorrect sentences.



The Grammar is Correct!

Sementic Analysis

In this stage, the input text is now *checked for meaning*, and every word and phrase is checked for meaningfulness.

For example:

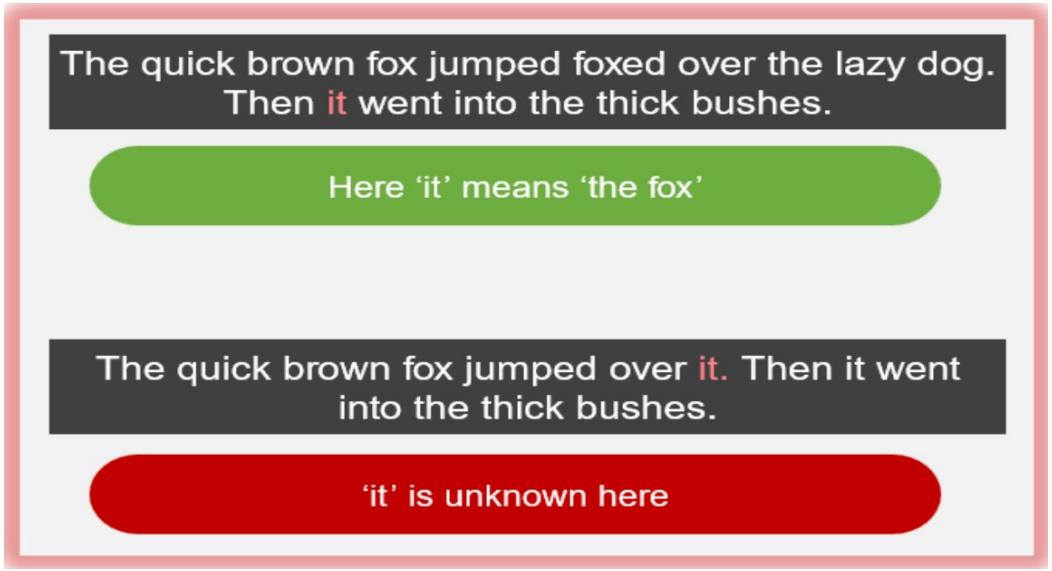
It will reject a sentence that contains ‘hot ice cream’in it. The fox jumped into the dog.



Sentences make actual sense!

Discourse integration

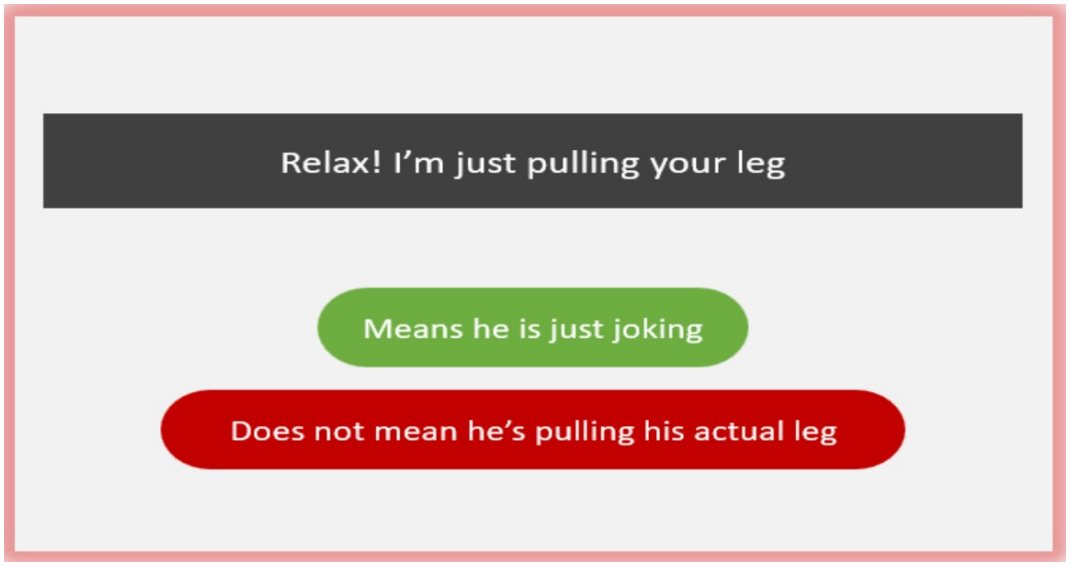
It is the *process of forming the story of the sentence*. Every sentence should have a relationship with its preceding and succeeding sentences



The flow of words makes sense!

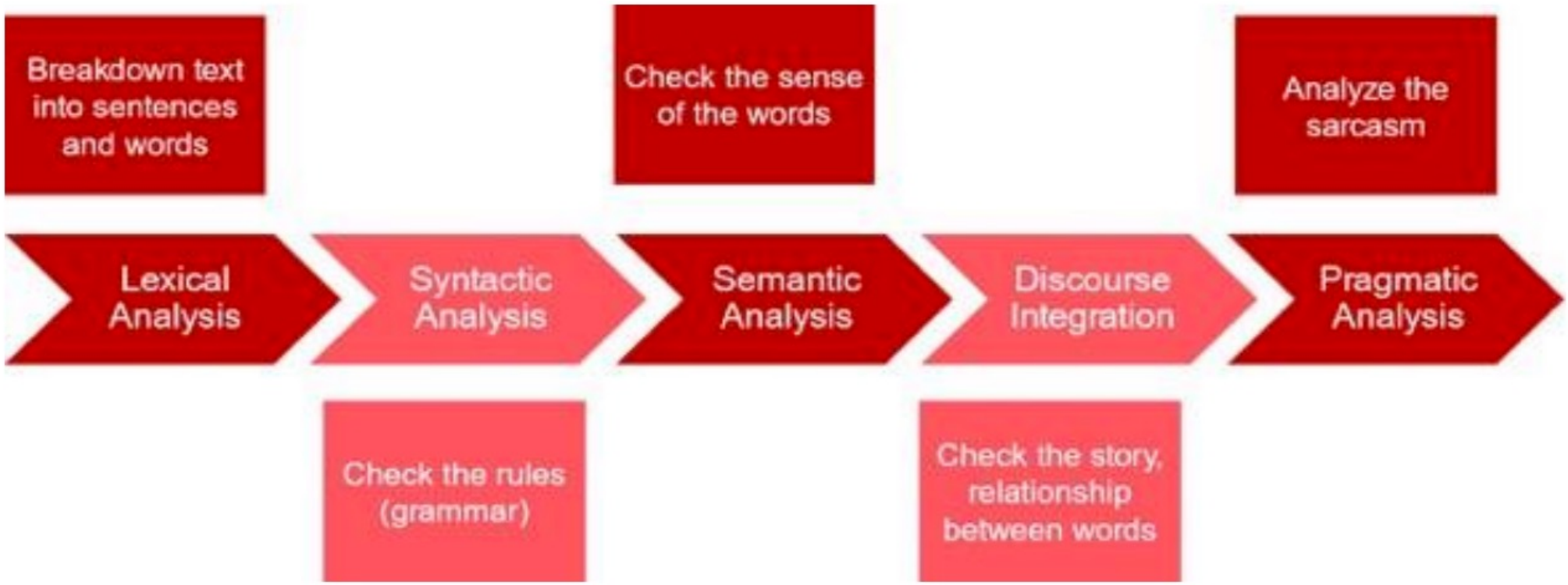
Pragmatic Analysis

In this stage, sentences are checked for their relevance in the real world. Pragmatic means practical or logical, i.e., this step requires knowledge of the intent in a sentence. It also means to discard the actual word meaning taken after semantic analysis and *take the intended meaning*.



The intended meaning has been achieved!

In summary,


Breakdown text into sentences and words
Check the sense of the words
Analyze the sarcasm
Lexical Analysis
Syntactic Analysis
Semantic Analysis
Discourse Integration
Pragmatic Analysis
Check the rules (grammar)
Check the story, relationship between words

Chatbots



One of the most common applications of Natural Language Processing is a chatbot. A chatbot is a computer program that's designed to simulate human conversation through voice commands or text chats or both. It can learn over time how to best interact with humans. It can answer questions and troubleshoot customer problems, evaluate and qualify prospects, generate sales leads and increase sales on an ecommerce site. There are a lot of chatbots available.

Purpose: Identify and interact with different chatbots.

Say: “Have you ever wondered why chatbots are created? They are meant to make it feel as if we are talking to a real human as this is the way we are comfortable with. There are several chatbots here. I will split you into groups. Spend some time interacting with the chatbot, and then we will review our experience.”

Let us try some of the chatbots and see how they work.

Elizabot - https://www.masswerk.at/elizabot/

Mitsuki - https://www.kuki.ai/

Cleverbot - https://www.cleverbot.com/

Singtel - https://www.singtel.com/personal/support

Script bot	Smart bot
• Script bots are easy to make. • Script bots work around a script which is programmed in them. • Mostly they are free and are easy to integrate to a messaging platform. • No or little language processing skills. • Limited functionality.	• Smart-bots are flexible and powerful. • Smart bots work on bigger databases and other resources directly. • Smart bots learn with more data. • Coding is required to take this up on board. • Wide functionality.

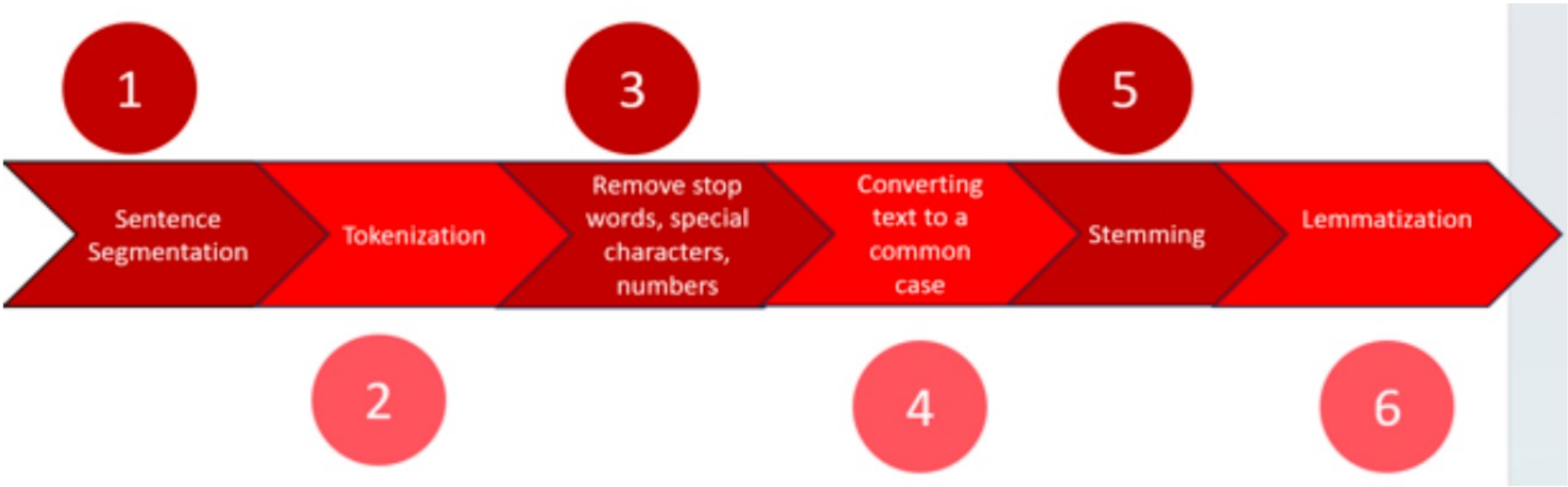
Text Processing



We all know that the language of computers is Numerical, the very first step that comes to our mind isto convert our language to numbers. This conversion takes a few steps to happen. The first step to it is Text Normalisation.

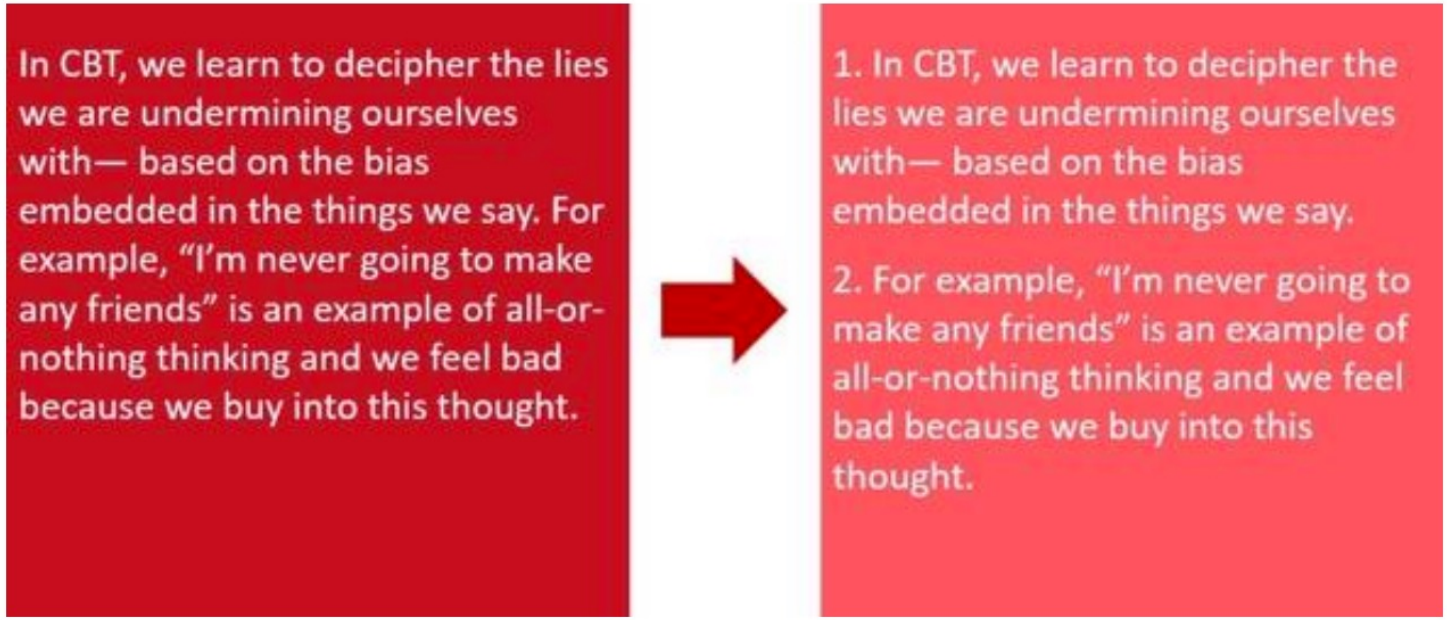
In Text Normalisation, we undergo several steps to normalise the text to a lower level. Before we begin, we need to understand that in this section, we will be working on a collection of written text. That is, we will be working on text from multiple documents andthe term used for the whole textual data from all the documents altogether is known as **corpus**.

STEPS OF THE NORMALISATION



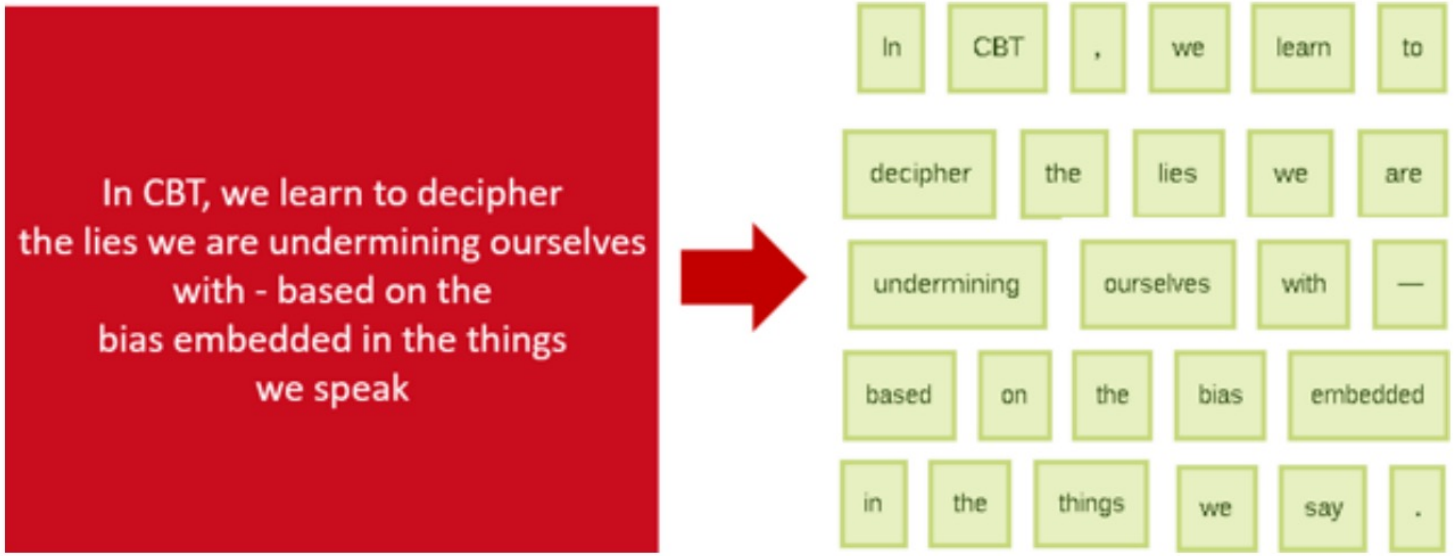
Sentence Segmentation

Under sentence segmentation, the *whole corpus is divided into sentences*. Each sentence is taken as a different data so now the whole corpus gets reduced to sentences.



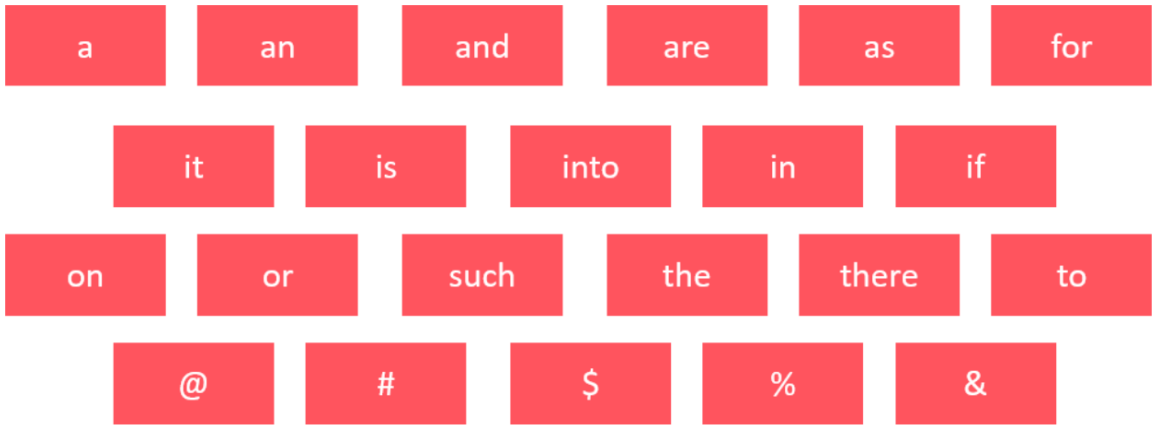
Tokenization

After segmenting the sentences, *each sentence is then further divided into tokens*. Tokens is a term used for any word or number or special character occurring in a sentence.



Removing Stop words, Special Characters and Numbers

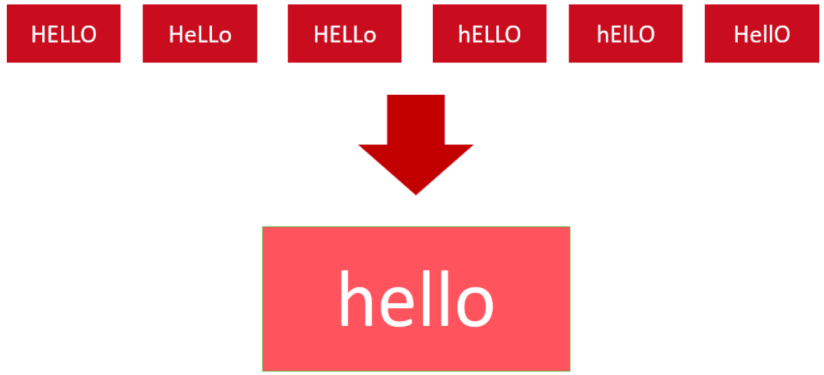
In this step, the *tokens which are not necessary are removed from the token list*. What are the possible words which we might not require?



These words occur the most in any given corpus but talk very little or nothing about the context or the meaning of it. Hence, to make it easier for the computer to focus on meaningful terms, these words are removed.

Converting text to a Common Case

After the stop wordsremoval, *we convert the whole text into a similar case*, preferably lowercase. This ensures that the case sensitivity of the machine does not consider the same words as different just because of different cases.



Here in this example, all the 6 forms of hello would be converted to lowercase and hence would be treated as the same word by the machine.

Stemming



In this step, the remaining words are reduced to their root words. In other words,stemmingis the process in which the affixes of words are removed and the words are converted to their base form.

Word	Affixes	Stem
healed	-ed	heal
healing	-ing	heal
healer	-er	heal
studies	-es	studi
studying	-ing	study

Note that in stemming, the stemmed words (words that we get after removing the affixes) might not be meaningful. Here in this example as you can see: healed, healing and healer all were reduced to heal but studies was reduced to studi after the affix removal which is not a meaningful word. ***Stemming does not take into account whether the stemmed word is meaningful or not.*** It just removes the affixes hence it is faster.

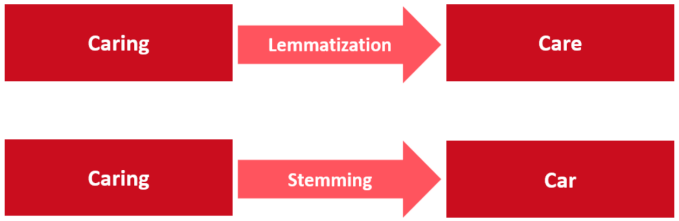
Lemmatization

VERY IMPORTANT

Stemming and lemmatization both are alternative processes to each other as the role of both the processes is same – removal of affixes. But the difference between both of them isthat in lemmatization, the word we get after affix removal (also known as lemma) is a meaningful one.Lemmatization makes sure that a lemma is a word with meaning and hence it takes a longer time to execute than stemming.

Word	Affixes	Stem
healed	-ed	heal
healing	-ing	heal
healer	-er	heal
studies	-es	**study**
studying	-ing	study

The difference between stemming and lemmatization can be summarized by this example:

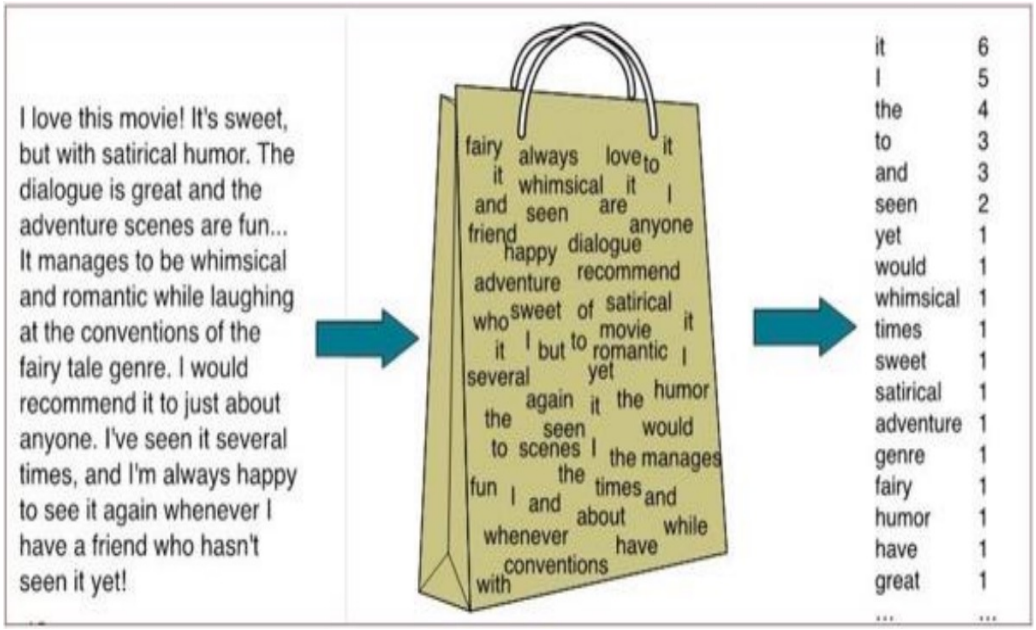


Bag of Words

VERY IMPORTANT

Algorithm

Bag of Words is a Natural Language Processing model which helps in extracting features out of the text which can be helpful in machine learning algorithms. In the bag of words, we get the occurrences of each word and construct the vocabulary for the corpus.



Thus, we can say that the bag of words gives us two things:
1. *A vocabulary of words for the corpus*
2. *The frequency of these words* (number of times it has occurred in the whole corpus).

Here calling this algorithm a "bag" of words symbolizes that the sequence of sentences or tokens does not matter. In this case, all we need are the unique words and their frequency.

Implementing BOW (step by step)

Here is the step-by-step approach to implementing the bag of words algorithm:

Step 1: Collecting data and pre-processing it.

Document 1: Aman and Avni are stressed
Document 2: Aman went to a therapist
Document 3: Avni went to download a health chatbo

Step 2: Create a Dictionary
Go through all the steps and create a dictionary i.e., list down all the words which occur in all three documents:
Dictionary:

aman	and	avni	are	stressed	went
download	health	chatbot	therapist	a	to

Note that even though some words are repeated in different documents, they are all written just once as while creating the dictionary, we create the list of unique words.

Step 3: Create a document vector

In this step, the vocabulary is written in the top row. Now, for each word in the document, fit matches the vocabulary, put a 1 under it. If the same word appears again, increment the previous value by 1. And if the word does not occur in that document, put a 0 under it.

aman	and	avni	are	stressed	went	to	a	therapist	download	health	chatbot
`1	1	1	1	1	0	0	0	0	0	0	0

Since, in the first document, we have words: aman, and, avni, are, stressed. So, all thesewords get a value of 1 and the rest of the words get a 0 value.

Step 4: Create document vectors for all the documents.

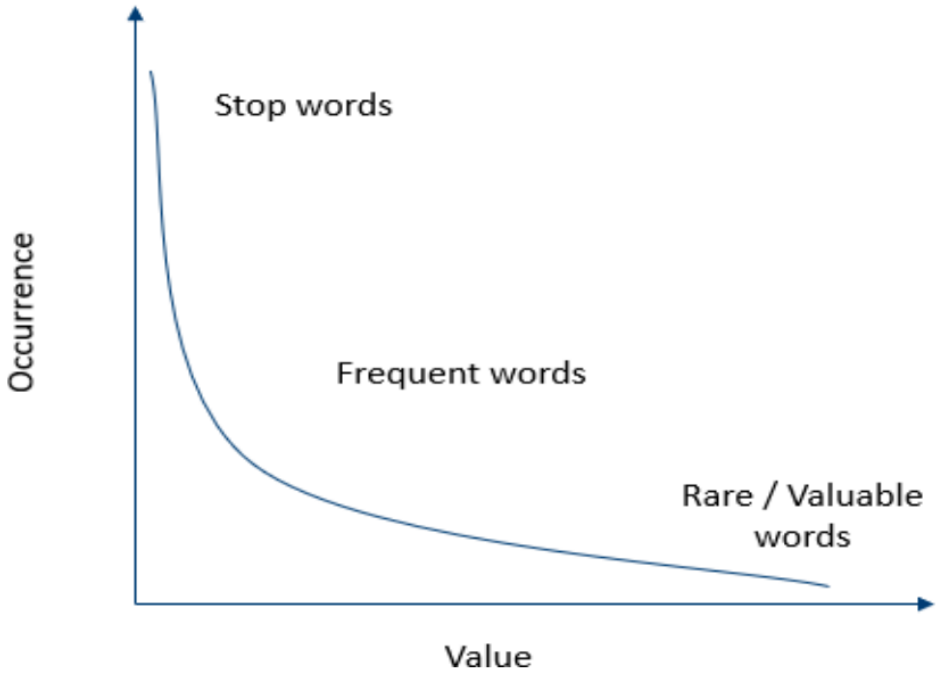
The same exercise hasto be done for all the documents. Hence, the table becomes:

aman	and	avni	are	stressed	went	to	a	therapist	download	health	chatbot
`1	1	1	1	1	0	0	0	0	0	0	0
1	0	0	0	0	1	1	1	1	0	0	0
0	0	1	0	0	1	1	1	0	1	1	1

In this table, the header row contains the vocabulary of the corpus and three rows correspond to three different documents. Take a look at this table and analyse the positioning of 0s and 1s in it. Finally, this gives us the **document vector table** for our corpus.

However, the tokens have still not been converted to numbers. This leads us to the final step of our algorithm: TF-IDF.

Term Frequency - Inverse Document Frequncy (TF-IDF)





And, this, is, the, etc. are the words which occur the most in almost all the documents. But these words do not talk about the corpus at all. Though *they are important for humans as they make the statements understandable to us, for the machine they are a complete waste as they do not provide us with any information regarding the corpus*. Hence, these are termed as ***stop words*** and are mostly removed at the pre-processing stage only

Term Frequency - TF

Term frequency is the frequency of a word in one document. Term frequency can easily be found in the document vector table as in that table we mention the frequency of each word of the vocabulary in each document.

aman	and	avni	are	stressed	went	to	a	therapist	download	health	chatbot
`1	1	1	1	1	0	0	0	0	0	0	0
1	0	0	0	0	1	1	1	1	0	0	0
0	0	1	0	0	1	1	1	0	1	1	1

Here, you can see that the frequency of each word for each document has been recorded in the table. These numbers are nothing but the ***Term Frequencies***!

Inverse Document Frequency - IDF

Document Frequency is the number of documents in which the word occurs irrespective of how many times it has occurred in those documents.

aman	and	avni	are	stressed	went	to	a	therapist	download	health	chatbot
`2	1	2	1	1	2	2	2	1	1	1	1

Here, you can see that the documentfrequency of ‘aman’, ‘avni’, ‘went’, ‘to’ and ‘a’ is 2 as they have occurred in 2 documents. The rest of them occurred in just one document hence the document frequency for them is 1.

Talking about inverse document frequency, we need to put the ***document frequency in the denominator*** while the ***total number of documents is the numerator***. Here, the total number of documents is 3, hence inverse document frequency becomes:

aman	and	avni	are	stressed	went	to	a	therapist	download	health	chatbot
`3/2	3/1	3/2	3/1	3/1	3/2	3/2	3/2	3/1	3/1	3/1	3/1

Formula

$$TFIDF(W) = TF(W) * \log(IDF(W))$$



Note that the TF values are for each document while the IDF values are for the whole corpus. Hence, we need to multiply the IDF values to each row of the document vector table.

aman	and	avni	are	stressed	went	to	a	therapist	download	health	chatbot
`1*log(3/2)	1*log(3)	1*log(3/2)	1*log(3)	1*log(3)	0*log(3/2)	0*log(3/2)	0*log(3/2)	0*log(3)	0*log(3)	0*log(3)	0*log(3)
1*log(3/2)	0*log(3)	0*log(3/2)	0*log(3)	0*log(3)	1*log(3/2)	1*log(3/2)	1*log(3/2)	1*log(3)	0*log(3)	0*log(3)	0*log(3)
0*log(3/2)	0*log(3)	1*log(3/2)	0*log(3)	0*log(3)	1*log(3/2)	1*log(3/2)	1*log(3/2)	0*log(3)	1*log(3)	1*log(3)	1*log(3)

aman	and	avni	are	stressed	went	to	a	therapist	download	health	chatbot
`0.176	0.477	0.176	0.477	0.477	0	0	0	0	0	0	0
0.176	0	0	0	0	0.176	0.176	0.176	0.477	0	0	0
0	0	0.176	0	0	0.176	0.176	0.176	0	0.477	0.477	0.477

Calculation - Explained

These numbers are the values of each for each document. Here, you can see that since we have less amount of data, words like ‘are’ and ‘and’ also have a high value. But as the IDF value increases, the value of that word decreases.

That is, for example:
Total Number of documents: 10
Number of documents in which ‘**and’** occurs: 10
Therefore, IDF(and) = 10/10 = 1
Which means: log(1) = 0. Hence, the value of ‘**and’** becomes 0.

On the other hand, the number of documents in which ‘**pollution**’ occurs: 3
IDF(pollution) = 10/3 = 3.3333…
This means log(3.3333) = 0.522;
which shows that the word ‘**pollution**’ has considerable value in the corpus.

Summary

Summarising the concept, we can say that:

1. Words that occur in all the documents with high term frequencies have the lowest values and are considered to be the stop words.

2. For a word to have a high TFIDF value, the word needs to have a high term frequency but less document frequency which shows that the word is important for one document but is not a common word for all documents.

3. These values help the computer understand which words are to be considered while processing the natural language. The higher the value, the more important the word is for a given corpus.

Applications of TF-IDF



TFIDF is commonly used in the Natural Language Processing domain. Some of itsapplications are:

Document Classification	Topic Modelling	Information Retrieval System	Stop word filtering
Helps in classifying the type and genre of a document.	It helps in predicting the topic for a corpus.	To extract the important information out of a corpus.	Helps in removing unnecessary words from a text body.

NLP - Case Walkthrough

Purpose: Students are introduced to the No-code tools for Natural Language Processing.

They will learn about Sentiment Analysis, one of the applications of NLP with the No-code tool Orange Data Mining. Learners will be able to understand this application with use cases

Examples of Code and No-code NLP Tools

Code NLP	No-Code NLP
NLTK package: Natural Language Tool Kit or NLTK is a package readily available for text processing in Python. The package contains functions and modules which can be used for Natural Language Processing.	**Orange Data Mining:** It is a machine learning tool for data analysis through Python and visual programming. We can perform operations on data through simple drag-and-drop steps.
SpaCy: SpaCy is an open-source natural language processing (NLP) library designed to build NLP applications. It offers various features such as tokenization, part-of-speech tagging, named entity recognition, dependency parsing, and more.	**MonkeyLearn:** MonkeyLearn is a text analysis platform that offers NLP tools and machine learning models for text analysis, supporting tasks such as classification, sentiment analysis, and entity recognition. Users can create custom models or use pre-trained ones for tasks like social media monitoring and customer feedback analysis.


THANK
YOU!